



UNIVERSIDAD
DE GRANADA



Inteligencia Artificial

Seminario 3: Búsqueda con Adversario (Juegos)

E.T.S. de Ingenierías Informática y de Telecomunicación
Universidad de Granada

*Este documento está protegido por la Ley de
Propiedad Intelectual ([Real Decreto Ley 1/1996 de 1
2 de abril](#)).*

*Queda expresamente prohibido su uso o
distribución sin autorización del autor.*

© Raúl Pérez

fgr@decsai.ugr.es

Departamento de Ciencias de la
Computación e Inteligencia Artificial
<http://decsai.ugr.es>

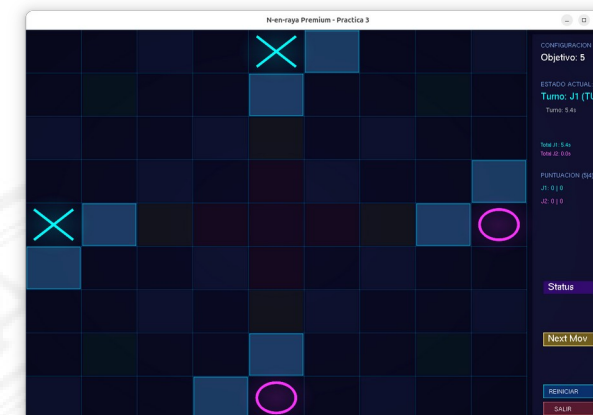


- **Introducción**
- Presentación del juego
- Presentación del Simulador
- Diseño y Desarrollo
- Método de evaluación/entrega de prácticas



En esta práctica diseñaremos e implementaremos técnicas de búsqueda con adversario en un entorno de juegos.

El juego elegido es una versión del conocido juego **3 en Raya** que llamamos **5 en Raya Táctico** (Modo Competición 9x9).



El objetivo es implementar los algoritmos **Status**, **Minimax** y **Poda Alfa-Beta**. También definir una **heurística** competitiva para el nuevo juego.



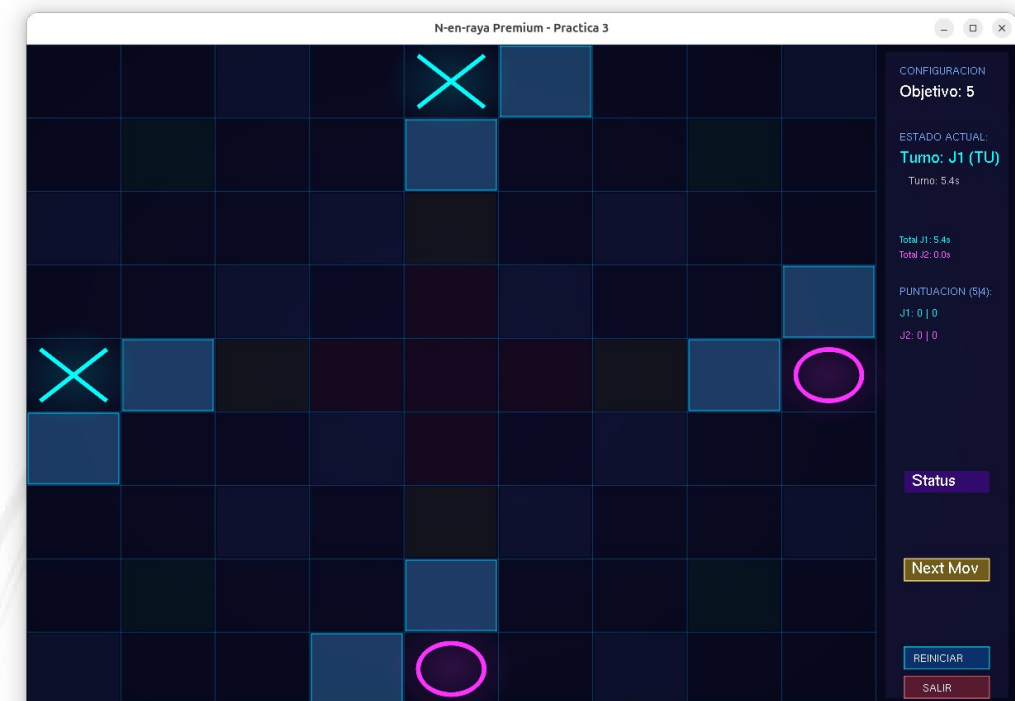
- Introducción
- **Presentación del juego**
- Presentación del Simulador
- Diseño y Desarrollo
- Método de evaluación/entrega de prácticas



Las Reglas del juego para el modo competición

1. Objetivo del Juego: Participan dos jugadores (J1 (X) y J 2 (O)). El objetivo es ser el primero en conseguir alinear 5 piezas consecutivas de su propio color en línea recta (horizontal, vertical o diagonal).

2. Tablero e Inicio: El tablero es de 9x9 casillas. En la posición inicial, hay 2 fichas de cada jugador ya posicionadas en el tablero como indica la figura.



3. Secuencia de Turnos (1-2-2-2): El flujo de colocación es asimétrico. En el Turno 1, el J1 coloca únicamente **1 pieza**. En todos los turnos sucesivos, los jugadores se alternan colocando un máximo de **2 piezas consecutivas** por turno.

Las Reglas del juego para el modo competición

4. La Regla de la Trinidad: El espacio de juego se divide en tres fases. Un jugador solo puede colocar una pieza en una casilla (fila, columna) si el cumple: $(\text{fila} + \text{columna}) \% 3 == \text{faseActual} \% 3$.

La faseActual empieza en 2 y avanza +1 cuando el jugador agota sus movimientos y cede el turno.

5. La Regla de Adyacencia: Toda pieza que se coloque en el tablero debe situarse obligatoriamente en una casilla vacía que sea **adyacente** (en cualquiera de las 8 direcciones) a una pieza ya existente.

6. ● Casillas Verdes (Místicas): Cuando una pieza aterriza en una casilla verde, el jugador gana de forma inmediata **+1 movimiento extra** para usar en su turno actual.

7. ● Casillas Rojas (Sabotaje): Estas casillas actúan como trampas. Si un jugador sitúa su pieza aquí, el tablero la absorberá y la transformará automáticamente en una pieza del **color del oponente**.

Las Reglas del juego para el modo competición

8. ● Casillas Amarillas (Bomba): Al colocar una pieza en esta celda, se produce una detonación que **elimina** (vuelve a dejar vacías) todas las celdas de la misma fila y la misma columna, respetando únicamente la posición de la bomba.

9. Resolución por Empate (Tie-break): Si el tablero se satura por completo sin que nadie logre el objetivo, el sistema otorgará la victoria a quién posea más líneas de 4 piezas. Si también tienen las mismas, la partida se dará por empatada.

Cuando no se juega en el modo Competición

Cuando se lanza el juego en configuraciones distintas, como puede ser 3x3 alineando 3, o 5x5 alineando 4, o cualquier otra a (9x9, alineando 5), las reglas del juego son las conocidas, es decir, turnos alternos de un único movimiento, donde cualquier de los jugadores puede colocar su fija en cualquier casilla que esté libre.





- Introducción
- Presentación del juego
- **Presentación del Simulador**
- Diseño y Desarrollo
- Método de evaluación/entrega de prácticas

Instalación del Simulación

Para instalarlo, debe seguir los pasos especificados en el GitHub <https://github.com/ugr-ccia-IA/practica3> de la asignatura.

El método es semejante al que se usó en la instalación de la práctica 2.

Prerrequisitos

Crear una cuenta en [GitHub](#).

Para ello, puedes usar tu correo personal, el de `@correo.ugr.es` o el de `@go.ugr.es`.

1. Añadir tu clave SSH a GitHub

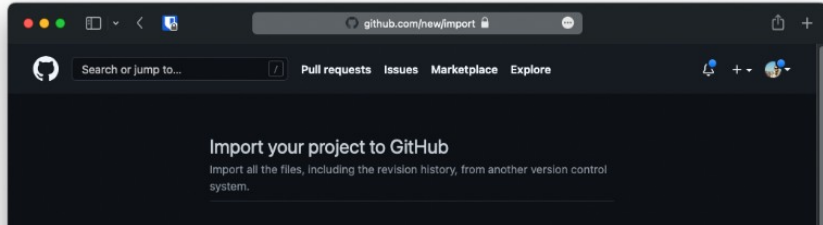
Hay varias maneras de conectarte desde tu ordenador a GitHub. Si utilizas un navegador, usarás tu usuario y contraseña. Desde el terminal, lo más cómodo es utilizar una clave SSH. Puedes crear una nueva si no tienes, o reutilizar una ya existente. Tienes toda la información para realizar la configuración en: [Conectar a GitHub con SSH](#)

2. Crear tu copia personal del repositorio de la asignatura

Cada estudiante debe tener su propia copia del repositorio para poder trabajar sobre ella. En adelante, a tu copia la llamaremos *origin*, y al repositorio original de la asignatura lo llamaremos *upstream* (NOTA: Estas son convenciones que la mayoría de los desarrolladores usan, pero los puedes llamar como quieras).

La forma usual de crear tu copia del repositorio es realizando un *fork*. Sin embargo, dado que realizar un *fork* de un repositorio con visibilidad pública obliga al que la copia sea también pública, nosotros usaremos un procedimiento diferente que nos permite que nuestra copia del repositorio sea privada.

Para realizar la copia, una vez que tengas creada tu cuenta en GitHub, haz click en <https://github.com/new/import> y rellena tal y como se ve en la imagen de abajo. El repositorio que quieres importar es `https://github.com/ugr-ccia-IA/practica2`. ¡Asegúrate de que tu repositorio es privado!



Suggested workflows

Based on your tech stack

- C/C++ with Make** [Conf](#)
Build and test a C/C++ project using Make.
- CMake based, multi-platform projects** [Conf](#)
Build and test a CMake based project on multiple platform.
- MSBuild based projects** [Conf](#)
Build a MSBuild based project.

[More workflows](#) [Dismiss su](#)

Parámetros de n_en_raya

n_en_raya / n_en_rayaSH es un simulador que ha de lanzarse desde línea de comandos. Su descripción de parámetros es la siguiente:

Definición de Jugadores (-p1 y -p2)

- * **humano** (por defecto): El jugador es controlado por una persona a través de la interfaz gráfica o la consola.
- * **aleatorio**: El agente elige cualquier casilla válida de forma totalmente al azar.
- * **minimax**: Activa el agente con búsqueda MiniMax (implementado por el estudiante).
- * **inteligente**: Activa el agente con Poda Alfa-Beta (implementado por el estudiante).
- * **status**: Activa la búsqueda exhaustiva completa para encontrar el veredicto teórico (Gana/Pierde/Empata).
- * **ninja1, ninja2, ninja3, ninja4**: Agentes pre-entrenados con distintos niveles de dificultad que actúan como “rivales” a batir.

Parámetros de n_en_raya

n_en_raya / n_en_rayaSH es un simulador que ha de lanzarse desde línea de comandos. Su descripción de parámetros es la siguiente:

Configuración del Tablero (-f, -c, -n)

- * **-f <int>**: Número de filas. (Defecto: 9).
- * **-c <int>**: Número de columnas. (Defecto: 9).
- * **-n <int>**: Número de piezas consecutivas necesarias para ganar. (Defecto: 5).

IMPORTANTE: Cambiar estas dimensiones afecta directamente al juego.

Solo en el **Modo Competición (9x9 con n=5)** se aplican las reglas de Turnos, Trinidad y Adyacencia descritas anteriormente. Para el resto de configuraciones, los turnos son alternos de un movimiento, no hay ni regla de Adyacencia, ni de Trinidad.

Parámetros de n_en_raya

n_en_raya / n_en_rayaSH es un simulador que ha de lanzarse desde línea de comandos. Su descripción de parámetros es la siguiente:

Parámetros de la IA (-d, -t, -id1, -id2)

- * **-d <int>**: Profundidad máxima de la búsqueda. (Defecto: 4).
- * **-t <double>**: Límite de tiempo en segundos por movimiento. (Defecto: 100.0).
- * **-id1 <int> / -id2 <int>**: Selector de heurística. Permite elegir qué función de evaluación usar para el J1 o J2 respectivamente.

IMPORTANTE: Obligatoriamente, en la **posición 0** de las heurísticas tiene que estar **heuristicaPrueba** y en la **posición 1**, la **heurística final** que elijáis para enfrentarse a los ninjas.

Parámetros de n_en_raya

n_en_raya / n_en_rayaSH es un simulador que ha de lanzarse desde línea de comandos. Su descripción de parámetros es la siguiente:

Otros parámetros

- * **-h**: Muestra el mensaje de ayuda con un resumen de todas las opciones.
- * **-nogui**: Desactiva la interfaz gráfica. La partida se ejecuta a máxima velocidad y el tablero se imprime por consola en cada turno.
- * **-file <nombreFichero>**: Carga un estado de tablero desde un fichero de texto. Más información sobre esto la podéis encontrar en el guion.

EJEMPLOS de n_en_raya

Ejemplo A: Entrenamiento contra Ninja de nivel 2

```
./n_en_raya -p1 humano -p2 ninja2 -f 9 -c 9 -n 5
```

Ideal para practicar estrategias contra un oponente de nivel medio antes de programar tu propia IA.

Ejemplo B: Experimento de Profundidad (Modo Texto)

```
./n_en_raya -p1 inteligente -id1 1 -p2 inteligente -id2 1 -d 3 -t 10.0 -nogui
```

Enfrenta a la misma IA contra sí misma a profundidad 7. Al no haber interfaz, la partida se resolverá rápidamente y verás el log de nodos visitados en la terminal.

EJEMPLOS de n_en_raya

Ejemplo C: Comparativa de Heurísticas (Modo Competición)

`./n_en_raya -p1 inteligente -id1 1 -p2 inteligente -id2 2 -d 4 -nogui`

Duelo entre tu heurística (ID 1) y tu heurística básica (ID 2). Es la mejor forma de validar científicamente si tus mejoras en la función de evaluación son realmente efectivas.

Ejemplo D: Resolución Completa (Tablero 3x3)

`./n_en_raya -p1 status -f 3 -c 3 -n 3 -nogui`

Intenta resolver el 3-en-raya clásico. Lo verás por consola.

Ejemplo E: Lucha contra Ninja3 (Modo Competición)

`./n_en_raya -p1 inteligente -id1 1 -p2 ninja3 -nogui`

Combate con el Ninja3 jugando como jugador 1 usando la heurística 1.

La Interfaz Gráfica

El Tablero de Juego

El tablero es una rejilla de tamaño variable (por defecto 9x9). En el caso de estar en el modo de competición (9x9,n=5) aplica las normas descritas anteriormente.

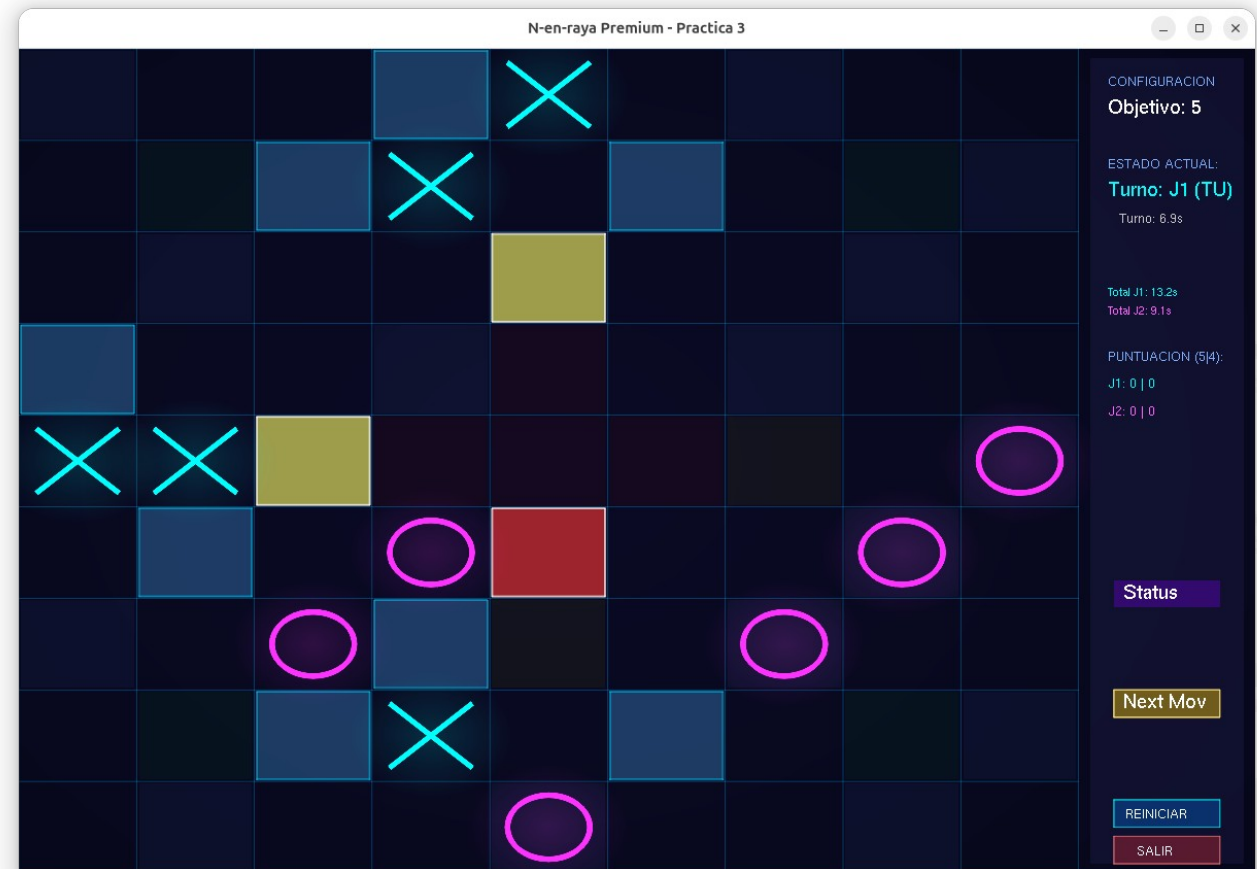
Elementos Visuales

Fichas Cian(X): **J1**

Fichas Magenta(O): **J2**

Línea Ganadora: Cuando un

Jugador gana se dibuja una línea gruesa translúcida uniando las piezas que consiguen la victoria.

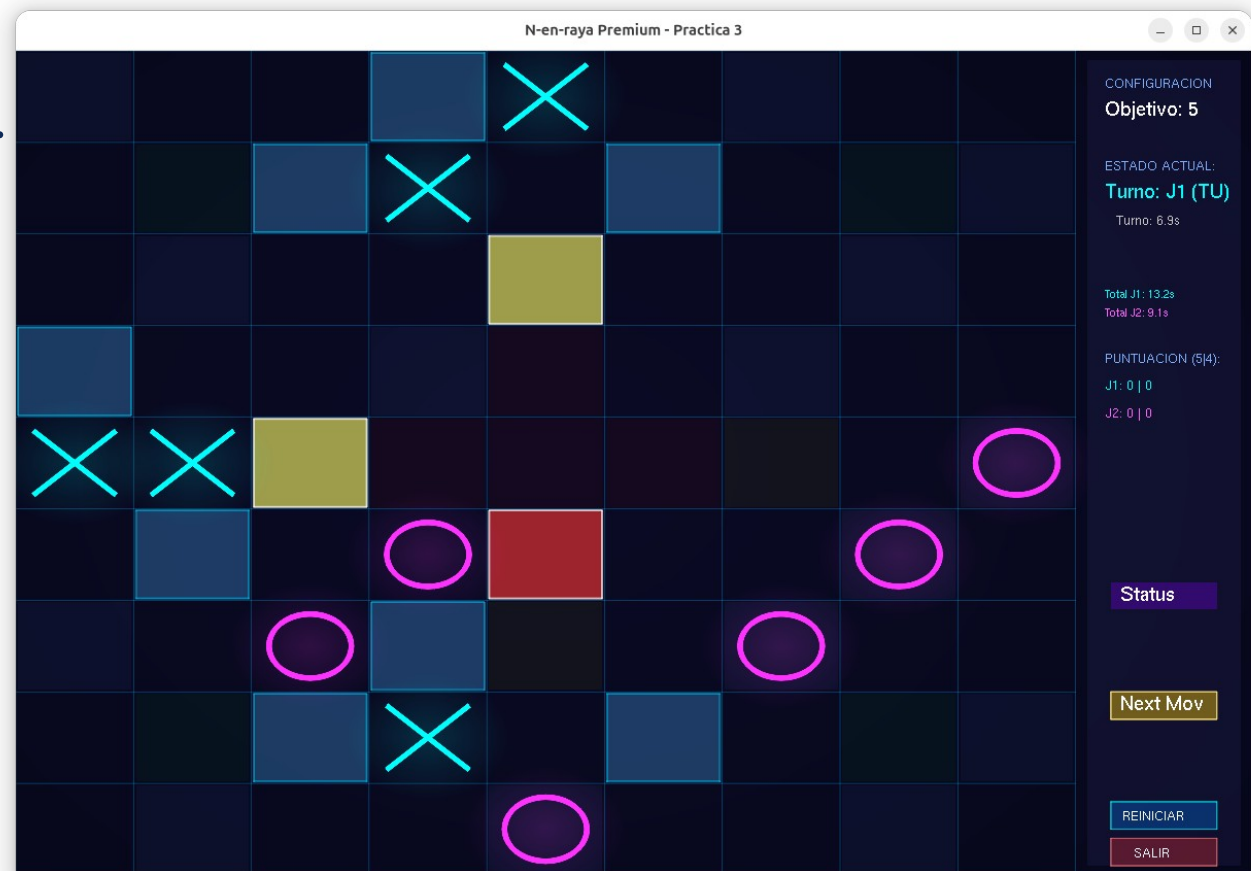


La Interfaz Gráfica

Tipos de Casillas

- **Negras/Azul oscuro:** Casillas normales.
- **Verdes:** Permiten repetir turno.
- **Rojas:** Colocan una ficha del oponente
- **Amarillas:** Limpian la fila y columna al ser activadas.

Las Verdes, Rojas y Amarillas solo existen en el modo de competición.



La Interfaz Gráfica

Botones de Control

● Botón **Next Mov** (Sugerencia)

Este botón es una herramienta de asistencia y depuración para el jugador humano. Al pulsarlo, el sistema invoca una instancia temporal de la IA del estudiante. Esta IA analizará el tablero con la profundidad configurada (-d) y realizará el mejor movimiento posible automáticamente.

● Botón **Status** (Análisis Completo)

Herramienta de análisis estratégico del software. Inicia una búsqueda exhaustiva (Full Search) para resolver el juego desde el estado actual.

Feedback Visual: Una vez calculado, mostrará sobre el botón el veredicto (ej: GANA J1 o EMPATE) y las coordenadas de la casilla que lleva a ese resultado.

● Botón **REINICIAR**

Limpia el tablero por completo. Resetea los cronómetros de tiempo acumulado. Vuelve el turno al Jugador 1.

● Botón **SALIR**

Cierra la aplicación de forma segura.

CONFIGURACION

Objetivo: 5

ESTADO ACTUAL:

Turno: J1 (TU)

Turno: 5.4s

Total J1: 5.4s

Total J2: 0.0s

PUNTUACION (5|4):

J1: 0 | 0

J2: 0 | 0

Status

Next Mov

REINICIAR

SALIR



- Introducción
- Presentación del juego
- Presentación del Simulador
- **Diseño y Desarrollo**
- Método de evaluación/entrega de prácticas

Pasos para construir la práctica

El Objetivo de la práctica es implementar 3 algoritmos (Status, Minimax y Poda Alfa-beta) y además definir una heurística que sea capaz de competir con los ninjas sobre el juego 5 en Raya Táctico en Tablero de 9x9.

Para ello, el estudiante solo puede modificar el archivo `AgenteEstudiante.cpp/.hpp` que se encuentra en la carpeta de `Comportamiento_Agente`. Dentro de estos archivos debe incluir las implementaciones que se piden.

DATOS MIEMBROS DE LA CLASE

- **id** (int): Indica que jugador qué es (1 o 2)
- **profundidadMax** (int): Indica la profundidad máxima en la búsqueda
- **tiempoMaxSegundos** (double): Limite máximo de tiempo para realizar un movimiento.
- **numHeuristica** (int): Selecciona la heurística a usar en el modo inteligente.
- **modo**: Modo de operación del agente (ALEATORIO, MINIMAX, INTELIGENTE, STATUS).

Pasos para construir la práctica

METODOS Y FUNCIONES YA IMPLEMENTADOS

- `std::pair<int, int> think(const Tablero& tablero)`
 - Método al que invoca el simulador para que el agente devuelva una jugada. Ya se encarga de llamar a Status, Minimax o alfaBeta según la elección en línea de comandos.
- `std::pair<int, int> JuegaAleatorio(const Tablero& tablero)`
 - Implementa un jugador aleatorio.
- `std::pair<int, int> JuegaInteligente(const Tablero& tablero)`
 - Se encarga de llamar a la Poda alfaBeta.
- `double heuristica(const Tablero& tablero)`
 - Método que agrupa las llamadas a todas las heurísticas que se definan. Se encarga de aplicar la heurística eligida en línea de comandos por el usuario.
- `double heuristicaPrueba (const Tablero& tablero)`
 - Función que implementa una heurística para probar los métodos minimax y alfaBeta. No se puede modificar y siempre debe ser la heurística 0.
- `std::pair<int, int> SacarMovimiento(const Tablero& padre, const Tablero &hijo)`
 - Función instrumental que permite sacar el movimiento realizado. Se le pasa el tablero padre y el hijo y devuelve el movimiento con el par de enteros fila, columna.

Pasos para construir la práctica

METODOS Y FUNCIONES **POR IMPLEMENTAR**

- Resultado **Status**(const Tablero &tablero, std::pair<int,int> &Mov)
 - Método de la clase donde el estudiante debe implementar el método status.
- double **minimax**(const Tablero &tablero, int profundidad, int prof_Max, std::pair<int,int> &Mov)
 - Método de la clase donde el estudiante debe implementar el método minimax
- double **alfaBeta**(const Tablero &tablero, int profundidad, int prof_Max, double alfa, double beta, std::pair<int,int> &Mov)
 - Método de la clase donde el estudiante debe implementar el método alfaBeta.
- double **heuristica1** (const Tablero& tablero)
 - Método donde el estudiante debe implementar una heurística que sea capaz de ganar al mayor número de ninjas posible. Puede definir tantas funciones como requiera o necesite. Debe recordar que la que usará para la competición contra los ninjas debe tener asociado la posición 1.

EJEMPLO DE IMPLEMENTACIÓN

`std::pair<int, int> JuegaAleatorio(const Tablero& tablero)`

Usaremos la implementación de `JuegaAleatorio`, un método ya implementado en `AgenteEstudiante.cpp` para ilustrar algunos métodos relevantes para la implementación.

```

68  std::pair<int, int> AgenteEstudiante::JuegaAleatorio(const Tablero& tablero) {
69
70      // Calculo los tableros descendientes de tablero
71      auto sucesores = tablero.getSucesores();
72
73      // Si no tiene descendientes, paso el turno
74      if (sucesores.empty()) return {-1, -1};
75
76      // Elijo aleatoriamente uno de los descendientes
77      int elegido = rand() % sucesores.size();
78
79      // Saco el movimiento realizado comparando el tablero original con el elegido.
80      std::pair<int,int> Mov = SacarMovimiento(tablero, sucesores[elegido]);
81
82      return Mov;
83  }

```

El método `getSucesores`, devuelve en un vector todos los tableros descendientes del tablero dado y `SacarMovimiento`, saca el movimiento que se realizó.

IMPORTANTE: Revisa los métodos de la clase `Tablero` para implementar los algoritmos pedidos. En el guion tienes una descripción de cada método.



- Introducción
- Presentación del juego
- Presentación del Simulador
- Diseño y Desarrollo
- **Método de evaluación/entrega de prácticas**

Entrega

Se pide desarrollar un programa (modificando exclusivamente el código de los ficheros `AgenteEstudiante.cpp/.hpp`) que implemente los algoritmos de búsqueda con adversario Status, Minimax, Alfabeta y una heurística para ganar al mayor número de ninjas posibles.

También se desarrollará una memoria de prácticas en formato pdf que se incluirá en la entrega.

Estos tres ficheros deberán entregarse en la plataforma PRADO de la asignatura, en un fichero ZIP, que no contenga carpetas, de nombre `practica3.zip`.

La fecha de entrega será el 31 de Mayo antes de las 23:00 horas

Entrega

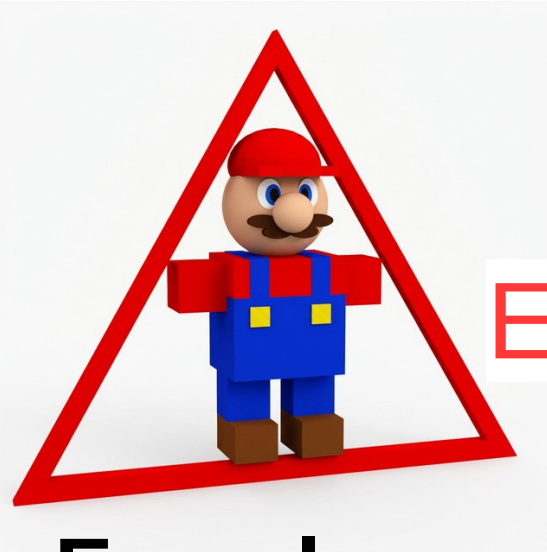
- La memoria de prácticas debe ser un documento de pocas páginas (**¡sin código!**) en formato PDF que, como mínimo, contenga los siguientes apartados:
 - Listado y breve descripción de las heurísticas.
 - Tabla comparativa de todas (al menos de las más relevantes) las heurísticas propuestas con los ninjas y entre ellas.
 - Análisis de los resultados obtenidos.
 - Opcionalmente, se puede incluir una reflexión personal sobre la práctica.

Evaluación

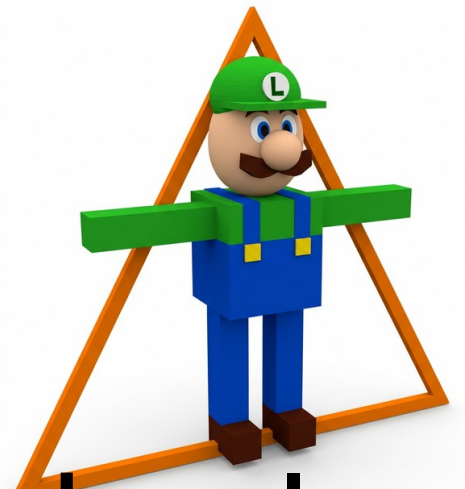
La práctica se califica numéricamente de 0 a 10. Se evaluará como la suma de los siguientes criterios:

- **La memoria de prácticas se evalúa de 0 a 2.** Se evaluará la claridad de las explicaciones así como la originalidad de las heurísticas contempladas.
- **La correcta implementación de los métodos Status, minimax y alfabeta se evalúa de 0 a 2.**
- **La eficacia del algoritmo se evaluará de 0 a 6 puntos** y estará basado en competir frente a cuatro jugadores ninja tanto de jugador 1 como de jugador 2. Con estas 8 partidas (4 como primer jugador y 4 como segundo jugador) se definirá la capacidad de la heurística desarrollada por el estudiante. La calificación será de 1 punto por cada victoria y de 0.5 puntos por cada empate.

En la competición contra los ninjas, la profundidad máxima será 4 con minimax y 7 con alfaBeta.



Esta práctica es **INDIVIDUAL**



En el caso de detectar prácticas copiadas, los involucrados (tanto el que se copió como el que se ha dejado copiar) tendrán suspensa la asignatura.

FECHA IMPORTANTE

Domingo 31 de Mayo antes de las 23:00 horas